APP DEVELOPMENT PROJECT REPORT

App Dev Project Report

1. Student Details

Name: Arya Mukherjee
Roll Number: 24F1002156
About Me: I am a student at IITM BS Degree Program with deep interest in software development and product build and problem solving. I enjoy build real-world application and meaningful contribution to real world projects.

---

2. Project Details

Project Title: Trekking Management Application
Problem Statement:
Adventure organizations require efficient systems to manage trekking activities involving trek organizers, staff, and participants. Currently, many trekking groups rely on spreadsheets, phone calls, or manual coordination, which makes it difficult to manage trek approvals, track bookings, avoid overbooking, and maintain trek history.
Build a **Trekking Management Application** web application that allows **Admin, Trek Staff, and Users (Trekkers)** to interact with the system based on their roles.
Approach:
The application was developed using a decoupled architecture, with **Flask** serving as the backend REST API framework utilizing a modular Blueprint structure, and **Vue.js 3** as the frontend Single Page Application (SPA) for a highly reactive user experience. It incorporates a SQLite database using **SQLAlchemy** for robust data management, and **JSON Web Tokens (PyJWT)** for secure, stateless role-based access control (Admin, Staff, User). The responsive frontend UI is styled with **Bootstrap 5** and communicates with the backend via Axios, while **Celery and Redis** are used to handle asynchronous background tasks like email notifications and CSV report generation.

---

3. AI/LLM Declaration

I used ChatGPT (GPT-5) and Gemini (pro) to assist in generating boilerplate Flask and Vue.js code, API endpoint documentation, UI scaffolding (Vue.js components and Bootstrap), and improving code readability and consistency. The extent of AI/LLM usage is approximately 30%, limited to code suggestions, documentation assistance, and development guidance. All final application logic, feature implementation, testing, debugging, database integration, and project customization were completed manually.

---

4. Technologies and Frameworks Used

Technology/Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Vue.js	A progressive JavaScript framework for building user interfaces. It enables a reactive, component-based frontend

	architecture for a seamless single-page application (SPA) experience.
Bootstrap5	Frontend styling and responsive design
PyJWT	Used for stateless authentication.
Axios	Used in the Vue frontend to make asynchronous API requests to the Flask backend, handling requests, responses, and interceptors seamlessly.
Celery with Redis broker	Used for offloading long-running or computationally heavy tasks to background worker processes, ensuring the main web server remains responsive.

---

5. Database Schema/ ER Diagram

Tables:

1. **Users** — stores user account details (id, name, email, password, phone, role, status, created_at)
2. **Treks** — stores trekking package details (id, trek_name, location, difficulty, slots, status, duration_days, staff_id, description)
3. **Bookings** — stores trek reservation details (id, user_id, trek_id, status)

Relationships:

One-to-Many → Users → Bookings
(One user can make multiple bookings.)
One-to-Many → Treks → Bookings
(One trek can have multiple bookings.)
One-to-Many → Users (Staff) → Treks
(One staff member can be assigned to manage multiple treks.)



---

6. **API Resource Endpoints**

The API is built using a RESTful pattern. Below is a high-level overview of the implementation:

Authentication Endpoints (/api/auth/*)

- Design: Handled via auth.py.
- Endpoints: /login, /register, /logout, /me.
- Implementation: Validates credentials, utilizes PyJWT to encode an authentication token, and decodes the token on subsequent requests to verify identity.

Admin Endpoints (/api/admin/*)

- Design: Handled via admin.py. Protected by a check_admin() middleware.
- Endpoints: Management routes for CRUD operations on Treks and Users/Staff.
- Implementation: Decodes JWT, ensures the user role is Admin. Returns JSON aggregated from SQLAlchemy ORM queries (e.g., retrieving all staff members). Trigger background tasks for CSV reports.

Staff Endpoints (/api/staff/*)

- Design: Handled via staff.py. Protected by a check_staff() middleware.
- Endpoints: Routes for viewing assigned treks and managing trek statuses (e.g., marking a trek 'Completed').
- Implementation: Filters Trek results strictly by staff_id matching the logged-in JWT user ID to ensure isolation of concern.

User Endpoints (/api/users/*)

- Design: Handled via user.py. Protected by a check_user() middleware.
- Endpoints: Searching open treks, creating bookings, cancelling bookings, updating profiles, and triggering asynchronous CSV exports.
- Implementation: Utilizes Redis caching on open treks to improve search performance. Booking endpoint programmatically enforces constraints (verifying slots > 0) within atomic database transactions.

---

7. **Architecture and Features**

The project follows a decoupled client-server architecture.

Backend Structure (/backend):

- Entry Point: app.py initializes the Flask app, configures the database, and registers Blueprints.
- Controllers/Routes (/routes): Segmented via Flask Blueprints (admin.py, auth.py, staff.py, user.py). Each module isolates logic relevant to that domain.
- Models (/models): Defines the SQLAlchemy database schemas.
- Background Processing: tasks.py stores Celery worker functions, while celery_worker.py bootstraps the worker process.

- Utilities: utils.py handles JWT generation/decoding, and cache.py manages Redis memory.

Frontend Structure (/frontend):

- Built using Vue.js. The logic is separated into views (/src/views/) mapped via a frontend Vue Router.

8. **Video Presentation**
   Drive Link:
   https://drive.google.com/file/d/1KqHXq3ytYgnra4AtsDfnWOQTmUeHF5Kb/view?usp=sharing

---
